

# TEXT ANALYTICS

## Text Processing and Linguistic Analysis

---

*Francesco Spinnato*

\* [francesco.spinnato@unipi.it](mailto:francesco.spinnato@unipi.it)  
University of Pisa



In Text Analytics, the primary input is **textual data**.

- Algorithms cannot directly operate on plain and dirty text: they require **structured representations** (e.g. vectors, probability distributions, word sequences).
- At its core, text can be viewed as a **sequence of characters**.
- Understanding text preparation and string manipulation (indexing, slicing, transformation) is the first practical step toward building text analytics pipelines.

avan, Mummie, there wouldn't be any railway fare and we shouldn't  
oms. Oh, *do* let us go in a caravan."

Mrs. Russell shook her head. "I know it sounds lovely, darling; but  
e we to get a caravan? It would cost at least fifty pounds to buy one,  
en if we had one, Daddy couldn't get away this summer. No, we  
ake up our minds to do without a holiday this year; but I'll tell you what  
ll do: we'll all go to Southend for the day, as we did last year, and  
r lunch and tea with us and have a splendid picnic."

"Then we can bathe again," said Bob; "but, oh! I do wish I could ha  
ny and ride," he added unexpectedly. "You don't know how I long  
ny," he continued, sighing deeply as he remembered the blissful holi  
en a friend let him share his little Dartmoor pony and ride occasionall  
"Southend is nothing but houses and people," cried Phyllis; "it's no b  
an this place; and oh! Mummie, I do so *long* for fields and flowers  
imals," she added piteously; and she shook her long brown hair forw  
hide the tears in her eyes.

"Never mind, darling, you shall have them one day," answered  
Russell with easy vagueness.

This really was not very comforting, and it was the most fortunate thing  
st at that moment a car stopped at the door.

"Uncle Edward!" shouted Bob, rushing from the room. Phyllis bru  
e tears so hastily from her eyes that she arrived at the front door almo  
on as he did, and both flung themselves on the tall, kindly-looking man st  
g beside the car.

"Uncle Edward! Uncle Edward!" they cried. "You've come at  
e've been longing to see you. Oh, how glad we are you're here!"

Now the delightful thing was that their uncle seemed just as pleased to  
em as they were to see him, and returned their hugs and greetings with  
most cordiality. They were just on the point of dragging him into  
ouse, hanging one on each arm, when he said: "Stop, not so fast. There  
me things to fetch in from the car."

So saying he began diving into the back of it and bringing out, not on  
itcase, but various parcels, which he handed out one by one.

"That's the pair of chickens I've brought for your mother," said he, ha

# Preparing Textual Data

Text documents are initially represented as sequences of characters, i.e. **strings**.

- Transformation is needed to convert text into sequences of words or **tokens**, i.e. meaningful sequences of characters.
- However, compound structures like *"New York"* need to be correctly identified as named entities.  
⇒ sophisticated linguistic processing is required for this task.

**Data preparation** (or preprocessing) involves:

- transformation of data into a form that can serve as the basis for analysis
- removal of disturbing noise

A **corpus** (plural *corpora*) is a collection of texts, gathered to be studied or to train a model.

- We write  $C$  for the corpus, and call a single text inside it a **document**  $d$ .
- Sizes vary enormously: a few hundred documents for a linguistic study, three trillion words for a corpus such as Dolma, built to train a language model.

A corpus is always a **sample** of a language, never the language itself. Which sample we take decides what anything built on it can learn, so it is worth asking where the text came from.

# Corpora: Where Did the Text Come From?

Five dimensions of variation, each of which changes what can be learned:

- **Language and variety:** over 7000 languages, most with regional and social varieties.
- **Genre:** newswire, fiction, Wikipedia, telephone conversations, legal text...
- **Demographics:** the age, gender and background of the writer leave traces.
- **Time:** language changes, and so do the meanings of individual words.
- **Form:** not every corpus is running prose. Code repositories, transcripts with speaker turns, treebanks\* and forum threads carry **structure** that a flat stream of tokens would discard.

A corpus should therefore travel with a **datasheet** recording why it was collected, in what situation, in which language variety, from which speakers, how it was sampled and annotated, and how it may be distributed.

---

\* Text corpus where every sentence is fully analyzed and labeled with its grammatical and syntactic structure

# Preliminaries

---

How many words are in the following sentence\*?

*"Pool rules matter because the pool has the entr-, uh, entrance blocked."*

- This sentence has **12 words** (counting *entr-* as one).
- If we count punctuation as separate "words", it has **15 words** (not counting the hyphen).
- If we don't count fillers like *uh* or fragments like *entr-*, it has **10 words**.
- If we count distinct words, it has **11 distinct words**.
- If we count distinct words and ignore upper/lowercase (case folding), it has **10 distinct words**.

---

\* More precisely, this is an utterance, i.e., the spoken correlate of a sentence.

A word can be considered in two different ways: **wordforms** and **lemmas**.

- **Wordform:** the complete, inflected or derived form of a word (e.g. "cats" is different from "cat").
- **Lemma:** the *citation form* (the one a dictionary files the word under). It groups the **inflected** forms of a single word:
  - *cat, cats* → *cat*;
  - *sing, sang, sung* → *sing*;
  - (Italian) *dorme* → *dormire*.

# Tokens

A **token** is a *unit of text*: one occurrence of whatever we have chosen to treat as a single item.

That choice is ours to make. A token may be

- a word,
- a punctuation mark,
- a piece of a word (sub-word, e.g. a morpheme),
- or even a single character.

If we take **words as tokens**, then the sentence has **12 tokens**.

*"Pool rules matter because the pool has the entr-, uh, entrance blocked."*

# Types and the vocabulary

*"Pool rules matter because the pool has the entr-, uh, entrance blocked."*

A **type** is a distinct token, counted once however often it occurs.

- This sentence has **11 (word) types**: ("*the*" is repeated twice).

The set of all types is the **vocabulary**, written  $V$ . Its size is  $|V|$

- so here  $|V| = 11$ .

$V$  is not fixed by the text alone.

- Case-fold it and *Pool* merges with *pool*, so  $|V|$  drops to 10.
- Both the unit we call a token and every normalization step later in this lecture reshape the vocabulary.

# The Lexicon

A **lexicon** is the list of the **lemmas** of a language, each stored with what we know about it: its part of speech, its inflected forms, sometimes its senses (different meanings).

It is worth keeping apart from the vocabulary  $V$  we just defined:

- $V$  is whatever happens to occur in **one corpus**. Change the corpus and  $V$  changes with it.
- A **lexicon** is knowledge **about a language**. It is built once, by hand or semi-automatically, and reused across corpora.

The two overlap but neither contains the other: a typo or a brand name sits in  $V$  and not in the lexicon, while most of the lexicon never appears in any given corpus.

# Morphemes: Parts of Words

A morpheme is the *minimal meaning-bearing unit* of a language. The study of morphemes is **morphology**.

- *fox* is one morpheme; *cats* is two (*cat* plus *-s* marking the plural).
- **Doc work-ed care-ful-ly wash-ing the glass-es**

Morphemes come in two classes:

- **Roots** carry the main meaning: *work*, *glass*, *dict*.
- **Affixes** add to it: *-ed*, *-es*, *pre-*.

**A lemma is not a root.** *Care*, *careful* and *carefully* share the root *care*, yet they are three separate lemmas with three dictionary entries.

- A lemma collapses **inflection** only;
- a root is what is left after stripping *every* affix, and it need not be a usable word on its own,
  - e.g., predict (*dict* is not an independent word).

# Text Representation

How do computers "see" text?

- For a computer, a text is stored as a **string**, i.e. a sequence of characters.
- Each character has a **position (index)**, usually starting from 0.
- This allows direct access to single elements of a string.
- Strings can be **sliced** to extract substrings.
- They provide a wide set of methods for manipulation (e.g. lowercasing, splitting, replacing).

Understanding strings is the first step toward text processing and indexing.

```
text = "piece of text"
text
# 'piece of text'

text[0]
# 'p'

text[0:3]
# 'pie'

len(text)
# 13
```

# Strings

In Python, a **string** is an *immutable, ordered* sequence of characters.

- **Immutable** means that once a string is created its content cannot be changed, and the characters are stored in a specific sequence.
- Strings can be created using either single quotes `' '` or double quotes `" "`.
- To create multiline strings, use triple double quotes `""" """` or triple single quotes `''' '''`.
- **Escape sequences** allow the insertion of quotes, newlines, tabs, or other special characters.

```
# Single or double quotes
```

```
a = 'this is a test'
```

```
b = "this is a test"
```

```
# Multiline string
```

```
c = """This is a  
multiline string"""
```

```
# 'This is a\nmultiline string'
```

```
# Escape sequences
```

```
quote = "He said: \"Python is fun!\""
```

```
newline = "First line\nSecond line"
```

```
tabbed = "Column1\tColumn2"
```

# Patterns

Every language gives strings the same basic operations: split them, join them, change their case, look for a **fixed** substring.

That is enough to find "*cat*", but not to answer any of these:

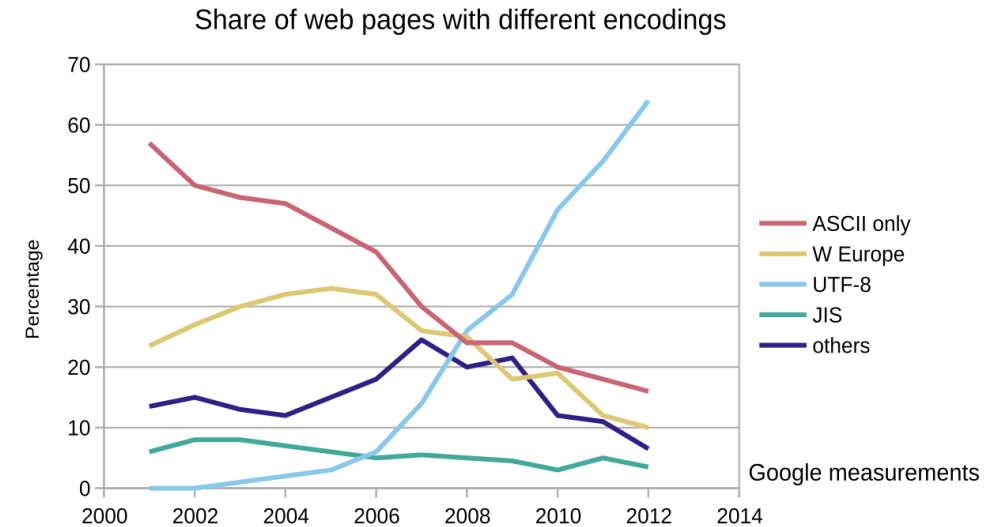
- find every **date** in a document, in any of the formats people write them
- strip out all **HTML tags**
- split on any run of **whitespace**, however long
- find every word ending in **-ing**
- check whether a string is a valid **email address**

Each asks for a **pattern**, not a literal string. **Regular expressions** (regex) are the notation for writing such patterns, allowing most of the techniques discussed in this lecture.

# Encodings

Text is represented using specific encodings.

- **Unicode** is a character encoding standard:
  - it assigns a number to each character
  - e.g. A → U+0041, 😊 → U+1F60A
- **UTF-8** specifies how those numbers are represented as bytes in a file or in memory.
  - € → U+20AC → E2 82 AC



Python supports multiple encodings when reading/writing text data. Other common encodings:

- **ASCII**: 7-bit encoding for English characters.
- **UTF-16**: 2 or 4 bytes per character.
- **ISO-8859-1**: encoding for Western European languages (Latin-1).

# Text Preprocessing

---

# Sentence Splitting

A corpus is a set of documents, and a document (usually) contains sentences.

**Sentence splitting** is the process of segmenting a text into distinct sentences, in order to treat each sentence as a basic unit.

Splitting a text into its sentences is not always a trivial task.

- Use of **punctuation** is the most relevant clue.
- But punctuation also reports other information (acronyms, numbers, initials...), so it can produce erroneous splits.

*E.g.* "I'll take the 8.30 train to St. Louis...or maybe I'll stay in LA. Time will tell!"

# Sentence Splitting

*E.g. "I'll take the 8.30 train to St. Louis...or maybe I'll stay in LA. Time will tell!"*

What a sentence splitter has to get right:

- **abbreviations:** *St., Dr., etc., approx.*
- **initials:** *J. R. R. Tolkien*
- **decimals and dates:** *8.30, 3.14*
- **ellipses:** *...or maybe I'll stay*
- **quoted sentences**, whose final punctuation sits inside the quotation marks
- **text with no final punctuation at all:** headlines, chat messages, transcripts

# Tokenization

**Tokenization** is the process of extracting units of text from a sequence of characters.

We've seen three candidates for tokens:

- words,
- morphemes,
- characters.

But each has problems as a unit. Words and morphemes seem approximately at the right level for NLP processing, since they tend to have consistent meanings, but they are challenging to define formally.

Characters are clearer to define, but seem too small a unit to choose for tokens.

Large Language Models (LLMs), such as GPT-3 and GPT-4, utilize a process called tokenization. Tokenization involves breaking down text into smaller units, known as tokens, which the model can process and understand. These tokens can range from individual characters to entire words or even larger chunks, depending on the model. For GPT-3 and GPT-4, a Byte Pair Encoding (BPE) tokenizer is used. BPE is a subword tokenization technique that allows the model to dynamically build a vocabulary during training, efficiently representing common words and word fragments. Although the core tokenization process remains similar across different versions of these models, the specific implementation can vary based on the model's architecture and training objectives.

In Western languages, words are often separated by whitespace and punctuation characters. A whitespace tokenizer is not language aware and is blind to "conventions" like abbreviations and contractions.

What makes tokenization hard:

- **clitics\*** and contractions: *don't* → *do* + *n't*, *l'amico* → *l'* + *amico*
- **hyphenation**: *state-of-the-art*, one token or four?
- **multiword expressions**: *New York*, *rock 'n' roll*
- **non-linguistic tokens**: URLs, emails, hashtags, emoji
- **no whitespace**: some languages (e.g. Chinese, Japanese, Thai) do not have spaces between words so word tokenization is much harder.
- **rich morphology**: a single Turkish or Finnish word may carry a whole clause\*\*, e.g., *talossanikin*: *talo* (house) + *ssa* (in) + *ni* (my) + *kin* (also/too)

---

\* morpheme that has syntactic characteristics of a word, but depends phonologically on another word or phrase

\*\* group of words that contains both a subject and a verb

# Tokens carry information

When working with tokenized text, we can perform various tasks to extract meaningful information:

- **Building a vocabulary:** identify all the unique terms used in the text, which helps in understanding the diversity of the language used.
- **Counting term frequency:** track how often each word appears. This shows which terms are most significant or central to the text's theme.
- **Visualizing frequency distribution:** plotting term frequency helps recognize patterns, such as the most frequent words, or identifying stopwords and rare words.
- **Tracking term occurrences:** determine where specific terms appear within the text, to understand their distribution and importance in different sections.
- **Analyzing contexts of terms:** understanding the context in which specific terms are used can reveal important relationships and nuances of meaning.

# Subword Tokenization

Splitting on whitespace leaves one problem unsolved: the **unknown word**.

If the corpus has *low*, *new* and *newer* but never *lower*, a word-level tokenizer meeting *lower* at test time has no entry for it: the best it can do is map it to a single `<UNK>` symbol, which throws away everything the word says.

**Subwords** solve the problem. Let the vocabulary hold units *smaller* than words, and every unseen word becomes a sequence of known ones:

$$\textit{lower} \longrightarrow \textit{low} + \textit{er}$$

- Most entries are still **whole words**; the rest are frequent **morphemes** and recurring substrings.
- A genuinely odd string, say an acronym like *NLP*, falls back to single characters.

In practice we usually use a data-driven approach to define tokens: the vocabulary is often **learned from a corpus**, not written by a linguist.

# Byte-Pair Encoding (BPE)

BPE is a data-driven approach to define tokens. BPE has a **trainer**, inducing a vocabulary from a corpus, and an **encoder**, applying it to new text.

Training starts from the set of all single characters, then repeats  $k$  times:

1. count every pair of **adjacent** symbols
2. **merge** the most frequent pair into a new symbol
3. add that symbol to the vocabulary

Merges never cross word boundaries, so the corpus is split on whitespace first, the leading space (  ) staying on the word.

# BPE: Example

Corpus:

set\_new\_new\_renew\_reset\_renew

s e t   \_ n e w   \_ n e w   \_ r e n e w  
\_ r e s e t   \_ r e n e w

Vocabulary:

\_ , e , n , r , s , t , w

Step 1 merges **n e** because it occurs 4 times, twice in *new* and twice in *renew*\*.

Both the corpus and vocabulary are updated:

Corpus:

s e t   \_ n e w   \_ n e w   \_ r e n e w  
\_ r e s e t   \_ r e n e w

Vocabulary:

\_ , e , n , r , s , t , w , n e

---

\* **e w** occurs 4 times too: ties are broken here by first occurrence.

# BPE: Example

By step 4 the algorithm has built `␣re`.

- Nobody told it that English has a word-initial prefix *re-*. It **discovered** the morpheme from adjacency counts.
- $k$  is the only parameter: the final vocabulary contains the characters plus  $k$  new symbols.
- BPE approximates morphology without a lexicon, in any language, from raw text.

| # | merge     | new token |
|---|-----------|-----------|
| 1 | n + e     | ne        |
| 2 | ne + w    | new       |
| 3 | ␣ + r     | ␣r        |
| 4 | ␣r + e    | ␣re       |
| 5 | ␣ + new   | ␣new      |
| 6 | ␣re + new | ␣renew    |
| 7 | s + e     | se        |
| 8 | se + t    | set       |

Vocabulary:

␣, e, n, r, s, t, w, ne, new,  
␣r, ␣re, ␣new, ␣renew, se, set

# Rule-Based (Language-Aware) Tokenization

BPE is the common choice, but what it hands back are **subwords**. For some applications the tokens have to be actual **words**. So instead of learning the units, we **pre-define a standard** and write rules for it.

For English we usually want to:

- split punctuation off, since commas help parsers and periods mark sentence boundaries
- but keep it **word-internally**: `m.p.h.` , `Ph.D.` , `AT&T` , `cap'n`
- keep prices, dates, URLs, hashtags and emails whole: `$45.55` , `01/02/06`
- expand **clitics**, parts of a word that cannot stand alone: `what're` → `what are`
- optionally glue **multiword expressions** into one token ( `New York` ), which needs a dictionary and ties tokenization to named entity recognition

Only a few of those rules survive a change of language. Such tokenizer is not neutral: it encodes a decision about *one* language's writing system.

# Normalization

Tokenization decides *where* to cut. **Normalization** decides which tokens should count as the **same type**.

| Variation                | Same type?         | Example                                  |
|--------------------------|--------------------|------------------------------------------|
| Case                     | usually            | <i>The, the</i>                          |
| Punctuation inside words | usually            | <i>U.S.A., USA, US</i>                   |
| Diacritics               | language-dependent | <i>café, cafe; (IT) perché, perche</i>   |
| Spelling variants        | usually            | <i>color, colour; organise, organize</i> |
| Whitespace and layout    | almost always      | double spaces, tabs, line breaks         |
| Numbers and dates        | task-dependent     | <i>1,000, 1000; 01/02/06, 2 Jan 2006</i> |

Every merge shrinks  $|V|$  and gives each remaining type more evidence, at the price of a distinction we can never recover later.

# Case Folding and Unicode

**Case folding** maps everything to lowercase. It is the cheapest normalization and often the only one applied.

- It helps whenever the writer's choice of case is not informative: *The* at the start of a sentence is the same word as *the* in the middle.
- It hurts whenever case *is* the signal: *US* (country) vs *us* (pronoun), *Apple* vs *apple*.

**Unicode normalization:** the same visible character can be stored differently.

- The *é* of *café* can be stored as one code point ( `U+00E9` ) or as two: a plain `e` ( `U+0065` ) followed by a combining acute accent ( `U+0301` ).
- The two strings look identical but they become two types.
- Unicode defines standard forms for this ( `NFC` composes, `NFD` decomposes).

Stemming and lemmatization, next, are the most aggressive normalizations: they merge whole **inflectional families** into one type.

# Stemming & Lemmatization

Sometimes you only care about stems or lemmas, not full words:

- A **stemmer** chops affixes with rewrite rules and does not need to know the word. What it returns is a **stem**, which need not be a real word.
- A **lemmatizer** looks the word up in a **lexicon** and needs its part of speech (e.g., noun, verb...). What it returns is the **lemma**, always a real dictionary entry.

| Word             | Stemmer       | Lemmatizer                               |
|------------------|---------------|------------------------------------------|
| <i>cars</i>      | <i>car</i>    | <i>car</i>                               |
| <i>studies</i>   | <i>studi</i>  | <i>study</i>                             |
| <i>was</i>       | <i>wa</i>     | <i>be</i>                                |
| <i>better</i>    | <i>better</i> | <i>good</i> (adj) / <i>better</i> (verb) |
| <i>operating</i> | <i>oper</i>   | <i>operate</i>                           |

# Stemming

Stemming aims at reducing the different inflections of a word to a common **stem**, by applying a set of language-dependent word transformation rules, which can result in a lexically incorrect or non-meaningful word.

**Advantages of stemming:** improved model performance, grouping similar words, easier to analyze and understand

**Disadvantages of stemming:**

- **Overstemming:** the stemmer reduces inflected words to the same stem even though they are not related (*universe* and *university* → *univers*)
- **Understemming:** the stemmer reduces inflected words to different stems, when they should be the same (*recognize* → *recogn*, *recognition* → *recognit*)
- **Language challenges:** as the target language's morphology, spelling and character encoding get more complicated, stemmers become harder to design.

Lemmatization aims at reducing the different inflections of a word to its **lemma**, using a deeper, and costlier, NLP analysis to reduce a word to its dictionary form. It identifies an inflected word's **lemma** based on its intended meaning.

## Advantages of lemmatization

- **Accuracy:** the analysis of words is conducted based on the word's part of speech, taking context into consideration.

## Disadvantages of lemmatization

- May need part of speech tagging
- Time-consuming.

# Zipf's Law, Stopwords and Rare Words

---

# How Big Is the Vocabulary?

Write  $|C|$  for the number of **tokens** in a corpus, against  $|V|$  for the number of **types**. The two grow at very different rates:

| Corpus                              | tokens $ C $ | types $ V $ |
|-------------------------------------|--------------|-------------|
| Shakespeare                         | 884 thousand | 31 thousand |
| Brown corpus                        | 1 million    | 38 thousand |
| Switchboard telephone conversations | 2.4 million  | 20 thousand |
| COCA                                | 440 million  | 2 million   |
| Google n-grams                      | 1 trillion   | 13 million  |

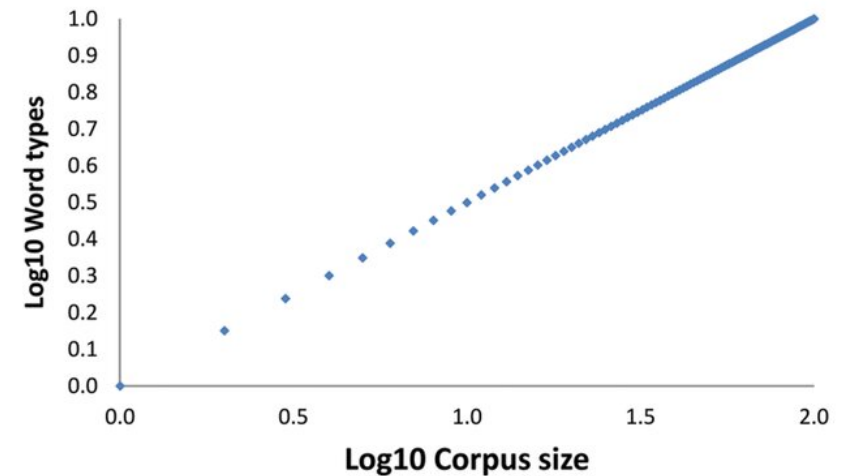
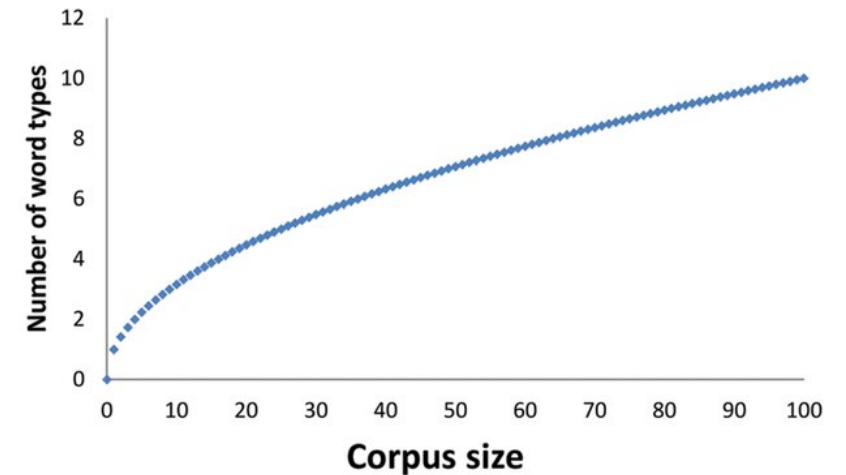
# Herdan's Law

Multiply the text by a thousand and the vocabulary grows by far less than that. The relationship is **Herdan's Law**, also known as **Heaps' Law**:

$$|V| = k |C|^\beta, \quad 0 < \beta < 1$$

with  $\beta$  usually between 0.44 and 0.56, depending on the size and genre of the corpus.

A corpus is therefore mostly **repetition**. What follows asks which words do the repeating, and what to do about those that never repeat at all.



# Too Many Features

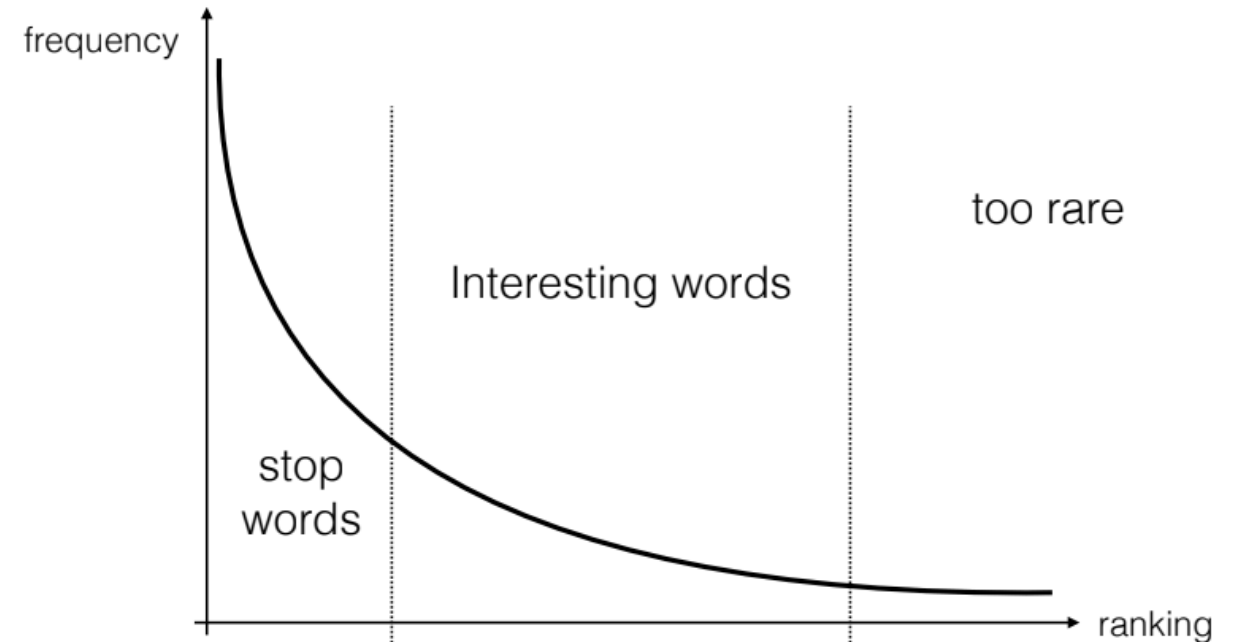
**Zipf's law:** the frequency of a word is inversely proportional to its rank in the frequency table.

- rank 1 (most frequent word) occurs about twice as often as rank 2,
- three times as often as rank 3, and so on...

$$f \propto 1/r$$

where  $f(r)$  is the frequency of the word with rank  $r$ .

## Zipf's Law



How much do words like *of*, *the*, *a*, *an*, *to*, ... contribute to the semantics of a document?

*the president of the united states of america*

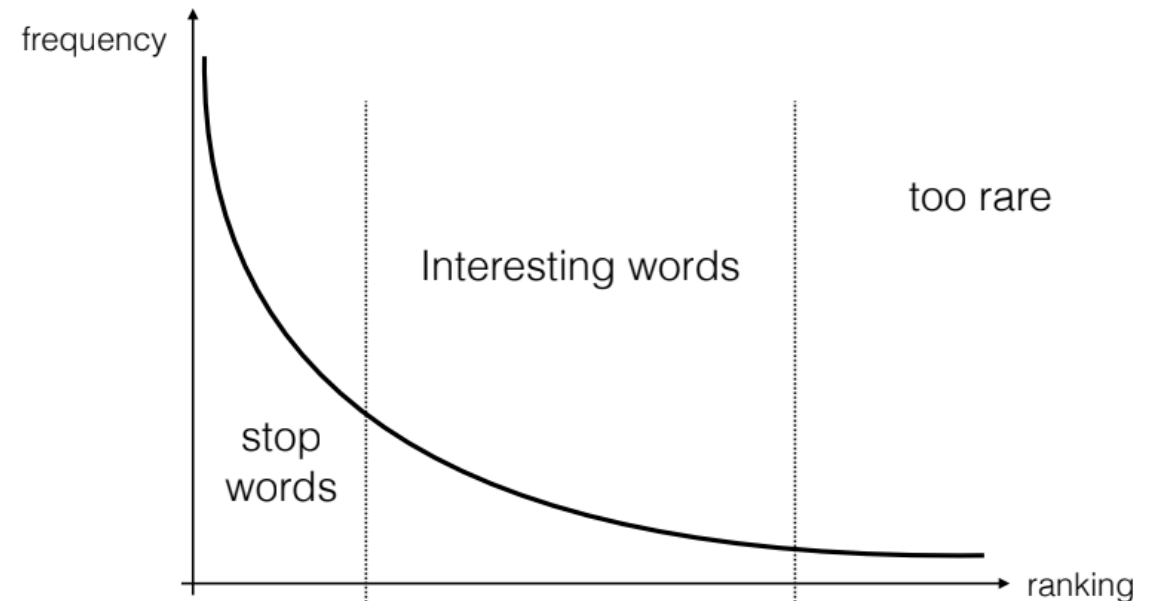
*president united states america*

The most common words of a language are usually referred to as **stopwords**: words that can be removed from text without losing much information.

Ready-made stopwords lists exist for most languages, yet they may not fit all applications.

A stopwords list is therefore a **task-dependent choice**, not a property of the language: a list that helps topic classification may destroy an authorship or sentiment task.

## Zipf's Law

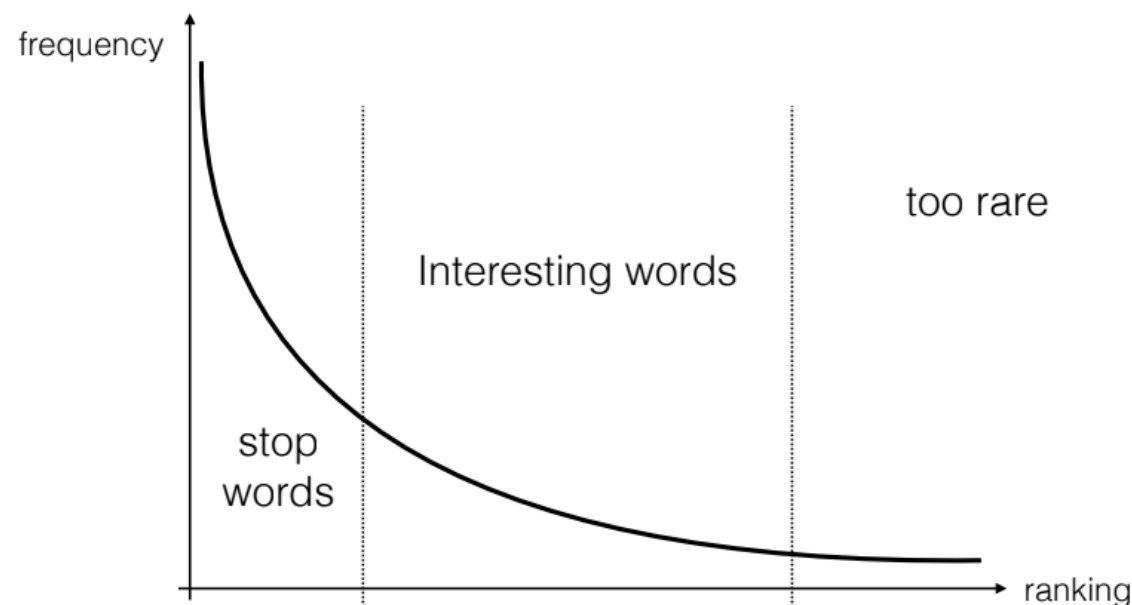


# Rare Features

Features that appear in very few documents may not bring useful information to generalize the text mining problem we are facing.

- If a word appears rarely in text observed in the past, it will likely be rare in future text.
- A word can be rare because it is a random **typo**.
- **Hapax legomena** (words that appear only once) usually account for a large portion of the distinct words appearing in a text collection.
- Removing rare words makes it faster to process the indexed text, and requires less memory space.

## Zipf's Law

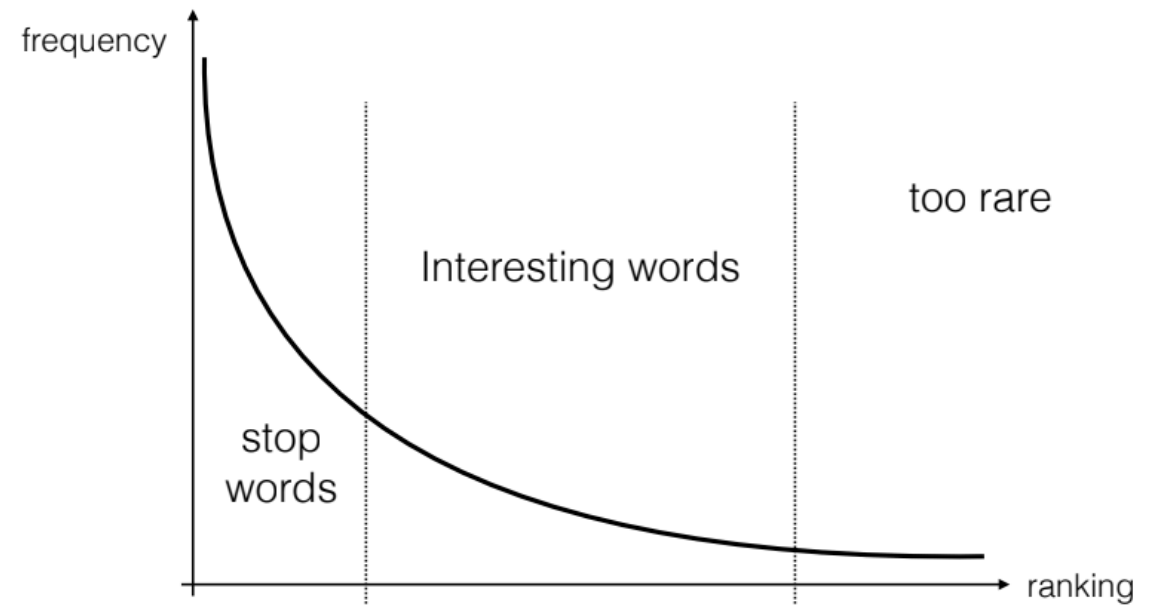


# Removing words

Removing stopwords and rare words has different effects on the corpus and the vocabulary sizes.

- Removing stopwords:
  - $|C|$  decreases a lot
  - $|V|$  decreases a little
- Removing rare words:
  - $|C|$  decreases a little
  - $|V|$  decreases a lot

## Zipf's Law



# Linguistic Annotation

---

# Linguistic Enrichment

**Linguistic enrichment** means adding linguistic information to prepared tokens: structured information about how words function and relate in context.

Once we have prepared tokens, we can enrich them with linguistic layers.  
Typical enrichment layers:

- Part of Speech (POS) tagging
- Morphological features
- Dependency parsing
- Named Entity Recognition (NER)
- Semantic relations (coreference, roles, etc.)

So we move from **isolated tokens** to **structured linguistic knowledge**.

**Part-of-speech (POS) tagging** is the process of assigning each term in a text to its part of speech.

- POS identify large classes of terms with similar syntactic roles: noun, verb, adjective, adverb, pronoun, preposition, conjunction, interjection, numeral, determiner.
- POS tagging can also identify more precise syntactic classes, such as common or proper names, and the mood of verbs.

⇒ Marking tokens with their part of speech **reduces ambiguity**:

*"I  $saw_1$  a bird" vs "Can you lend me a  $saw_2$ ?"*

$\text{semantic}(saw_1) \neq \text{semantic}(saw_2)$

**Which tags?** The tagset is a design choice:

- **Universal Dependencies** uses 17 coarse tags, designed to be comparable across languages.
- The **Penn Treebank** tagset uses 45 tags for English alone, distinguishing e.g. singular from plural nouns and verb tenses.

Taggers are trained on corpora that have been **manually annotated** with one of these tagsets.

# Morphological Analysis

**Inflectional features** describe form: Number, Gender, Case, Person, Tense, Mood, Aspect, Degree, ...

When performing POS tagging or morphological analysis, each token is annotated using a set of standardized labels. The most widely adopted framework is **UD (Universal Dependencies)**.

Different annotation standards exist, defining tagsets and morphological features used to label tokens consistently across languages:

- **UPOS** - 17 coarse-grained POS categories valid across all languages, e.g. **NOUN** , **VERB** , **ADJ** , **ADV** .
- **XPOS** - language-specific POS tags: fine-grained, language-dependent tagsets, often inherited from older national treebanks. E.g. English Penn Treebank tags: **NN** , **NNS** , **VBZ** , **VBG** , ... Italian ISDT: **S** , **A** , **V** , ... (spaCy reports them with the features attached: **S\_\_Gender=Fem|Number=Plur** )

# Morphological Analysis

Why morphological analysis matters:

- Improves **lemmatization and disambiguation** (e.g. "*porta*" as noun vs verb in Italian).
- Crucial for **morphologically rich languages** (IT, DE, FI, RU, ...), where word forms carry grammatical information such as case or gender.
- Improves downstream tasks like **NER** and **syntactic parsing**.
- Enables **feature engineering**, e.g. the proportion of `VERB[Past|Indicative]` , or noun gender agreement.

Morphological features are represented as token-level **attribute: value** pairs.

- e.g. `Number=Plur|Gender=Fem|Case=Nom` in UD.

Modern models often predict POS tags and morphological attributes **jointly**, using multi-task neural architectures.

# Named Entity Recognition (NER)

Named Entity Recognition is the task of detecting and classifying words or expressions that refer to real-world entities such as persons, organizations, locations, dates or numerical expressions.

- It transforms unstructured text into structured data, by identifying **who, what, where and when**.
- NER assigns a label (entity type) to each identified span of text. Typical entity types: PERSON, ORG, LOC, GPE, DATE, MONEY, EVENT, PRODUCT, ...

The screenshot displays a Named Entity Recognition (NER) interface. At the top, there is a legend bar with colored boxes and labels: Person (p, blue), Loc (l, yellow), Org (o, black), Event (e, green), Date (d, red), and Other (z, purple). Below the legend, a text snippet about Barack Obama is shown. Various spans of text are highlighted with colored boxes and labeled with the corresponding entity type from the legend. The labels are: 'Barack Hussein Obama II' (Person, blue), 'August 4, 1961' (Date, red), 'American' (Other, purple), 'the United States' (Loc, yellow), 'January 20, 2009' (Date, red), 'January 20, 2017' (Date, red), 'Democratic Party' (Org, black), 'African American' (Other, purple), 'United States Senator' (Other, purple), 'Illinois' (Loc, yellow), and 'Illinois State Senate' (Org, black). Each label is followed by a small 'x' icon.

Take the sentence:

*Jane Villanueva of United, a unit of United Airlines Holding*

A **span** is a contiguous run of tokens treated as a single unit: *United Airlines Holding* is one organization, three tokens long.

Finding spans is harder than tagging words, because two decisions are coupled:

- **segmentation**: where does each span begin and end? Most words belong to no entity at all.
- **typing**: which class is it? *Washington* may be a person, an organization, a place or a government.

# BIO Tagging

POS tagging has no segmentation problem, since every word simply gets one tag. **BIO tagging** removes it for NER too, by encoding the boundaries into the labels themselves, one label per token:

- **B-X** : this token **begins** a span of type X
- **I-X** : this token is **inside** a span of type X
- **O** : this token is **outside** any span

|       |            |    |        |   |   |      |    |        |          |         |
|-------|------------|----|--------|---|---|------|----|--------|----------|---------|
| Jane  | Villanueva | of | United | , | a | unit | of | United | Airlines | Holding |
| B-PER | I-PER      | O  | B-ORG  | O | O | O    | O  | B-ORG  | I-ORG    | I-ORG   |

A span ends whenever the next token is not I-X of the same type

# NER: Three Approaches

## Rule-based

Lists of known names (**gazetteers**) plus regex patterns.

- good for dates, codes, closed domains
- fast and transparent
- limited coverage, constant upkeep

## Statistical

Hand-designed features fed to a classifier.

- the word, its POS tag, capitalization, prefixes and suffixes
- the same for its neighbours
- still useful when data is scarce

## Neural (*current standard*)

No hand-designed features at all.

- the model learns each token's representation from its **context**
- a tagging layer predicts the BIO labels
- often trained jointly with POS and parsing

All three produce the same output, a BIO label per token. What changes is where the knowledge comes from: a human writing rules, a human choosing features, or the model learning both.

# NER: Challenges and Evaluation

- **Ambiguity:** "Apple" → organization vs fruit.
- **Domain adaptation:** models trained on news may fail on biomedical or legal text.
- **Multilinguality:** entity boundaries and capitalization differ across languages.
- **Nested entities:** "University of California, Berkeley" → ORG containing LOC.

NER is evaluated at the span level, not tokens, and per entity type. A system that should return *United Airlines Holding* but stops at *United* has made a real error, even though every token it labeled was inside an entity.



The first two spans match exactly. The third is a boundary error: right type, right starting word, but stopping two words early.

# Syntactic Constituency

**Constituency** is the idea that groups of words behave as single units, called **constituents**.

The classic kind of constituent is the **noun phrase**, written **NP** : a group of words built around at least one noun. Verb phrases ( **VP** ) and prepositional phrases ( **PP** ) work the same way.

- *the Broadway coppers,*
- *a high-class spot such as Mindy's,*
- *three parties from Brooklyn,*
- *they.*

What is the evidence that these words group together? **A single pronoun can stand in for the whole group**, however long it is. For example, *they* is the smallest noun phrase, the one every longer phrase can be exchanged for.

# Syntactic Constituency

| Whole phrase before the verb                         | Replaced by one pronoun |
|------------------------------------------------------|-------------------------|
| <i>three parties from Brooklyn arrive...</i>         | <i>they arrive...</i>   |
| <i>the Broadway coppers love...</i>                  | <i>they love...</i>     |
| <i>a high-class spot such as Mindy's attracts...</i> | <i>it attracts...</i>   |

Try the same swap on only *part* of a phrase and it fails:

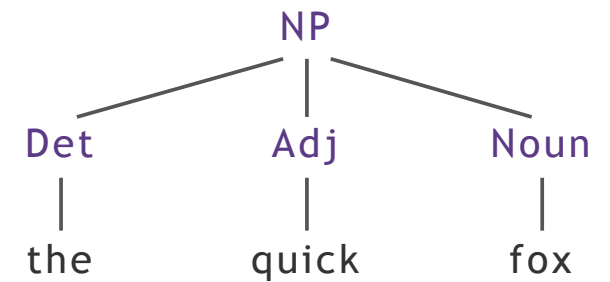
*three they arrive*      *three parties they arrive*

These substitution tests provide evidence that **certain sequences of words form constituents**. A grammar represents these sequences using categories such as noun phrase (NP), rather than listing every possible sequence of words.

# Context-Free Grammars and Parse Trees

A context-free grammar formally describes how constituents can be constructed. It is the complete set of rules describing which structures are allowed.

```
NP   → Det Adj Noun
Det  → the | a
Adj  → quick | lazy
Noun → fox | dog
```



Read the first rule as “a noun phrase can consist of a determiner, followed by an adjective, followed by a noun.” In the other rules, | means “or.”

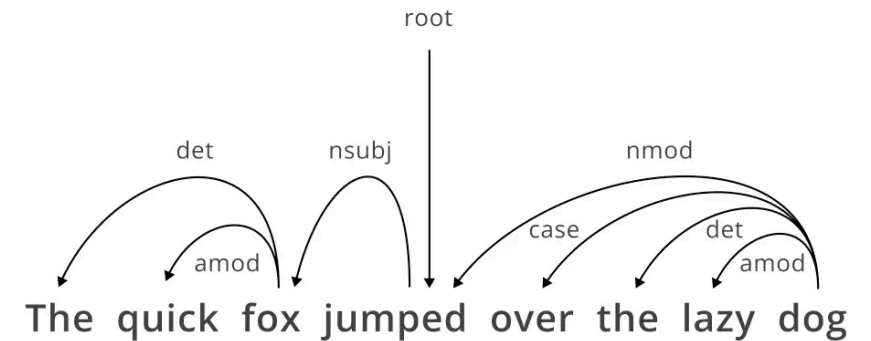
**Parsing** applies the rules in reverse: given *the quick fox*, find the tree the grammar could have produced.

# Dependency Parsing

A second way to describe the same syntax. A constituency tree groups words into **phrases**; a dependency tree links words **directly to one another**, with no phrase nodes at all.

- Every word points to its **head**, the word it depends on.
- One word is the **root**, usually the main verb, and depends on nothing.
- Every link is labeled with the relation it expresses: **nsubj** for the subject of a verb, **amod** for an adjective modifying a noun, etc.

## Dependency Parsing



Dependencies are the more common choice in practice. The labels read almost directly as *who did what to whom*, which is what the tasks built on top of parsing usually want.

# Chunking (Shallow Parsing)

Both trees above are expensive to build, and often more than a task needs. **Chunking**, or shallow parsing, keeps only the **flat phrases** of a sentence and builds no tree at all.

**[the little dog]<sub>NP</sub> [barked]<sub>VP</sub> [at]<sub>PP</sub> [the postman]<sub>NP</sub>**

Nothing nests, and nothing overlaps: every word belongs to at most one chunk. That flatness is the whole point, because it turns parsing back into **sequence labeling**, with the same BIO tags used for NER.

So no grammar and no search: the taggers that already handle POS and NER handle chunking too.

# The Preprocessing Pipelines

---

# Recap: Normalising the Text

| Step                 | What it does                      | The catch                                                        |
|----------------------|-----------------------------------|------------------------------------------------------------------|
| Sentence splitting   | cut the stream into sentences     | abbreviations, initials, decimals, no final punctuation          |
| Tokenization         | cut sentences into tokens         | clitics, hyphens, multiword names, scripts without spaces        |
| Subword tokenization | learn word pieces from the corpus | fixes unknown words; the vocabulary is learned, not written      |
| Normalization        | fold case, standardise word forms | destroys distinctions some tasks need ( <i>US</i> vs <i>us</i> ) |
| Stemming             | strip affixes by rule             | fast, but the output need not be a word                          |
| Lemmatization        | map to the dictionary lemma       | needs a lexicon and a POS tag                                    |
| Stopword removal     | drop the Zipf curve head          | task-dependent                                                   |
| Rare word removal    | drop the tail                     | hapax legomena are a large portion of $V$                        |

# Recap: Annotating the Text

| Layer                    | What it adds                             | How                                                              |
|--------------------------|------------------------------------------|------------------------------------------------------------------|
| POS tagging              | a word class per token                   | sequence labeling; UD has 17 tags, Penn Treebank 45              |
| Morphological analysis   | feature-value pairs per token            | <code>Number=Plur Gender=Fem</code> , predicted jointly with POS |
| Named entity recognition | typed spans: person, organization, place | sequence labeling with BIO tags                                  |
| Chunking                 | flat, non-overlapping phrases            | BIO again, on top of POS                                         |
| Constituency parsing     | a full phrase-structure tree             | a context-free grammar                                           |
| Dependency parsing       | head-dependent relations between words   | a labeled tree over the tokens themselves                        |

# What the Annotations Are For

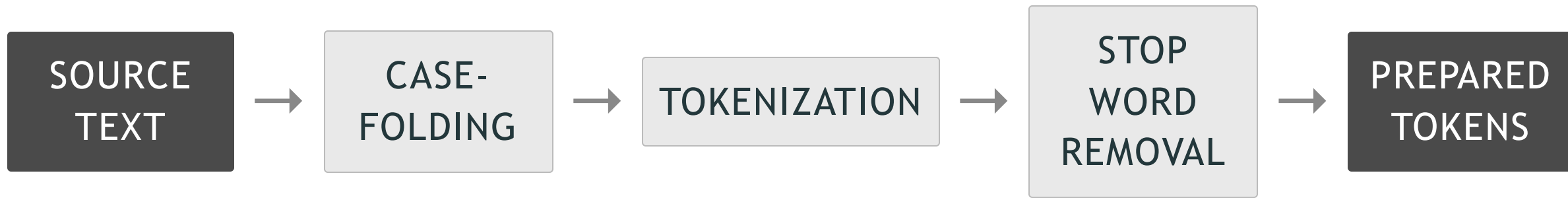
**Annotations** add structure to text, without yet asking what the text *says*. They are a first step toward:

- **Features for a model.** Annotations become numbers that can be visualized or consumed by a model:
  - proportion of verbs in the past tense
  - ratio of adjectives to nouns
  - frequency of named entities ( PERSON , ORG )
  - average sentence length in tokens
- **Information extraction:** turning prose into records a database could hold.

# (Pre)processing Pipeline

The processing pipeline transforms the original text into cleaned and normalized tokens.

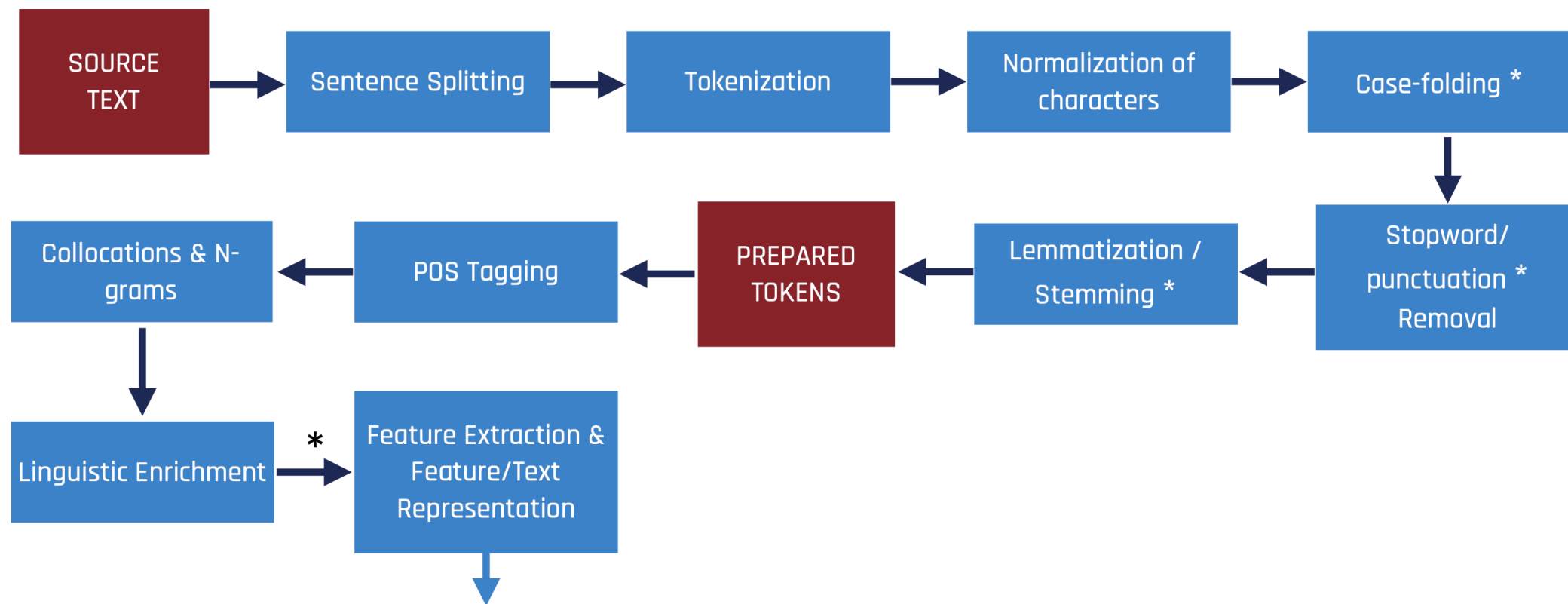
The simplest preprocessing pipeline consists of three steps: case-folding into lowercase, tokenization, and stop word removal.



Note: the pipeline may vary depending on the original text and the specific task to be performed.

# (Pre)processing Pipeline

A more sophisticated pipeline can include a **data cleaning** and a **linguistic preprocessing** phase.



We are still missing some of these steps!

## Appendix: Regular Expressions

---

# Regular Expressions

**Regular expressions** (REs or regexes) are a powerful tool for efficiently handling low-level text-processing tasks.

- They provide a concise syntax for defining text patterns and enable operations such as matching, searching and text manipulation.
- A regular expression describes a set of strings. It acts as a small **search language** based on explicitly defined rules.

## Why use regular expressions?

- **Automate repetitive tasks:** search, replace or extract recurring patterns.
- **Extract structured data from text:** retrieve useful information from logs, documents and other unstructured sources.
- **Validate data:** check whether a string has the expected structure.
- **Use a compact syntax:** express complex text-processing operations concisely.

# One Running Example: Application Logs

Throughout the lesson, we will build one expression for processing this application log:

```
2026-09-20 09:15:03 INFO user=anna action=login ip=192.168.1.10 status=OK
2026-09-20 09:17:41 WARNING user=annabelle action=login_attempt ip=192.168.1.25 status=NOT_OK
2026-09-20 10:02:18 INFO user=luca action=logout ip=10.0.0.8 status=OK
2026-09-20 10:05:07 ERROR user=guest action=password_reset ip=10.0.0.12 status=FAILED
2025-12-31 23:59:59 INFO user=anna action=login ip=172.16.0.4 status=OK
invalid log entry; datetime 0000-00-00 00:00:00; dump 192X168Y1Z10
```

Try each expression on [regex101.com](https://regex101.com) using the **Python** flavor and multiline mode.

# Key Features

Regular expressions are commonly used for searching, validating, extracting and transforming text.

## Key features include:

- Concatenation
- Metacharacters
- Anchors
- Quantifiers
- Alternation
- Character classes
- Negation
- Grouping
- Escape sequences
- Lookarounds

# Concatenation

**Concatenation** means placing two or more patterns (characters) next to each other. The resulting expression matches them consecutively in the same order.

```
INFO user=
```

matches `INFO` , followed by a space, followed by `user=` .

Although the result is one complete regular expression, it is constructed by concatenating smaller patterns.

Regular expressions are **case-sensitive** by default: `INFO` and `info` are different patterns.

# Metacharacters and Wildcards

**Metacharacters** have a special meaning in regular expressions.

The wildcard `.` matches any character except a newline:

```
192.168.1.10
```

This matches `192.168.1.10`, but it could also match `192X168Y1Z10` because each `.` is a wildcard.

# Metacharacters and Wildcards

To search for literal dots, escape them with `\`:

```
192\.168\.1\.10
```

Other examples:

| Pattern                 | Meaning                                 |
|-------------------------|-----------------------------------------|
| <code>INFO ERROR</code> | <code>INFO</code> or <code>ERROR</code> |
| <code>\.</code>         | a literal dot                           |
| <code>\ </code>         | a literal vertical bar                  |
| <code>\\</code>         | a literal backslash                     |

# Anchors

Anchors match positions rather than characters.

| Anchor          | Meaning                              | Log example                                                                  |
|-----------------|--------------------------------------|------------------------------------------------------------------------------|
| <code>^</code>  | start of a string or line            | <code>^2026</code> selects lines from 2026                                   |
| <code>\$</code> | end of a string or line              | <code>status=OK\$</code> selects lines ending in <code>status=OK</code>      |
| <code>\b</code> | word boundary                        | <code>\banna\b</code> selects <code>anna</code> , not <code>annabelle</code> |
| <code>\B</code> | position that is not a word boundary | <code>anna\B</code> selects <code>anna</code> inside <code>annabelle</code>  |

Compare:

- `anna` : This matches both `anna` and the beginning of `annabelle` .
- `\banna\b` : This matches only `anna` as a complete word.

Careful: `OK$` on its own would also select the `status=NOT_OK` line, which does end in `OK` . Anchoring the word, `\bOK$` , excludes it, because `_` is a word character and there is no boundary between `_` and `0` (next slide).

# Word Boundaries: A Subtle Case

Compare:

- `login` : This matches both `login` and the beginning of `login_attempt` .
- `\blogin\b` : This matches `login` , but not `login_attempt` .

Why? A word boundary exists between a word character and a non-word character. The underscore `_` is a **word character**, just like letters and digits. Therefore, there is no word boundary between `n` and `_` .

|                            |                       |
|----------------------------|-----------------------|
| <code>login action</code>  | boundary after n: yes |
| <code>login_attempt</code> | boundary after n: no  |

# Character Classes and Negation

Character classes match one character from a set:

| Pattern            | Meaning                                           |
|--------------------|---------------------------------------------------|
| <code>[abc]</code> | <code>a</code> , <code>b</code> or <code>c</code> |
| <code>[IEW]</code> | <code>I</code> , <code>E</code> or <code>W</code> |
| <code>[0-9]</code> | one digit                                         |
| <code>[A-Z]</code> | one uppercase ASCII letter                        |
| <code>[a-z]</code> | one lowercase ASCII letter                        |

For example, `[A-Z]+` finds sequences of uppercase letters such as `INFO` , `WARNING` and `ERROR` .

A caret at the beginning of a character class negates it: `[^a-z]` matches **one** character that is **not** a lowercase ASCII letter.

# Common Shorthand Character Classes

Regex provides aliases for commonly used character classes:

| RE              | Typical expansion                  | Meaning            |
|-----------------|------------------------------------|--------------------|
| <code>\d</code> | <code>[0-9]</code>                 | digit              |
| <code>\D</code> | <code>[^0-9]</code>                | non-digit          |
| <code>\w</code> | letters, digits and <code>_</code> | word character     |
| <code>\W</code> | the opposite of <code>\w</code>    | non-word character |
| <code>\s</code> | spaces, tabs and line breaks       | whitespace         |
| <code>\S</code> | the opposite of <code>\s</code>    | non-whitespace     |

# Quantifiers

Quantifiers specify how many times the preceding element may occur.

| Quantifier         | Meaning             | Example              |
|--------------------|---------------------|----------------------|
| <code>*</code>     | zero or more        | <code>\d*</code>     |
| <code>+</code>     | one or more         | <code>[A-Z]+</code>  |
| <code>?</code>     | zero or one         | <code>s?</code>      |
| <code>{n}</code>   | exactly $n$         | <code>\d{2}</code>   |
| <code>{n,m}</code> | between $n$ and $m$ | <code>\d{1,3}</code> |
| <code>{n,}</code>  | at least $n$        | <code>\d{2,}</code>  |
| <code>{0,m}</code> | at most $m$         | <code>\d{0,3}</code> |

- `\d{4}-\d{2}-\d{2}` matches a date
- `\d{2}:\d{2}:\d{2}` matches a time
- A first approximation of one IPv4 component is `\d{1,3}`
- `user=\w+` matches `user=anna` and `user=annabelle`.

Regex generally searches for the leftmost match, and greedy quantifiers consume as much as they can while still allowing the complete pattern to succeed.

# Alternation

The bar `|` expresses **alternation**: the expression can match the pattern on its left or the pattern on its right.

- `INFO|WARNING|ERROR` : This matches any of the three log levels.

Alternation has low precedence.

- `^INFO|ERROR$` : This means “ `INFO` at the beginning or `ERROR` at the end”. It does not mean “a complete line containing either log level”.

Parentheses make the intended alternatives explicit:

- `^(INFO|ERROR)$` : This means a complete line containing either `INFO` or `ERROR`

# Grouping

Parentheses `()` combine multiple elements into a single unit.

Without grouping:

- `ab+` : The `+` applies only to `b` : it matches `ab` , `abb` , `abbb` , ...

With grouping:

- `(ab)+` : The `+` applies to the complete group: it matches `ab` , `abab` , `ababab` , ...

In the log, grouping lets us describe a complete IP address:

```
\d{1,3}(\.\d{1,3}){3}
```

The group `\.\d{1,3}` must occur exactly three times.

# Capturing and Non-Capturing Groups

By default, parentheses create a **capture group**. Captured text can later be extracted or reused.

- `(INFO|WARNING|ERROR)` : The matched log level is stored as group 1.

Sometimes parentheses are needed only for structure. A **non-capturing group** uses `(?:...)` :

- `\d{1,3}(?:\.\d{1,3}){3}` : This still groups the repeated section, but does not store it as a separate captured value.

```
(?:INFO|WARNING|ERROR) user=(\w+)
```

Only the username becomes group 1.

# Escape Sequences

Special characters must be escaped with a backslash when they need to be matched literally.

| RE              | Matches                 |
|-----------------|-------------------------|
| <code>\.</code> | a literal period        |
| <code>\?</code> | a literal question mark |
| <code>\*</code> | a literal asterisk      |
| <code>\ </code> | a literal vertical bar  |
| <code>\\</code> | a literal backslash     |
| <code>\n</code> | a newline               |
| <code>\t</code> | a tab                   |

`ip=\d{1,3}(:|\.\d{1,3}){3}` contains both escaped dots and grouping. The dots between the IP components are literal characters, not wildcards.

# Building the Complete Pattern

Start with the date and time: `^\d{4}-\d{2}-\d{2}\s+\d{2}:\d{2}:\d{2}`

Add the log level and username:

```
^\d{4}-\d{2}-\d{2}\s+\d{2}:\d{2}:\d{2}\s+(?:INFO|WARNING|ERROR)\s+user=\w+
```

Then match the complete line:

```
^(\\d{4}-\\d{2}-\\d{2})\\s+(\\d{2}:\\d{2}:\\d{2})\\s+(INFO|WARNING|ERROR)\\s+user=(\\w+)\\s+action=(\\w+)\\s+ip=(\\d{1,3}(?:\\.\\d{1,3}){3})\\s+status=(OK|NOT_OK|FAILED)$
```

The anchors ensure that the **whole line** has the expected structure. Therefore, `invalid log entry` is not accepted.

# Capture Groups

The complete expression stores seven pieces of information:

| Group | Captured value | Example      |
|-------|----------------|--------------|
| 1     | date           | 2026-09-20   |
| 2     | time           | 09:15:03     |
| 3     | level          | INFO         |
| 4     | user           | anna         |
| 5     | action         | login        |
| 6     | IP address     | 192.168.1.10 |
| 7     | status         | OK           |

Groups are numbered by the position of their opening parenthesis, from left to right. The group inside the IP pattern is non-capturing: `(?:\.\d{1,3}){3}` . Therefore, it does not alter the numbering of the useful groups.

# Substitution

**Substitution** transforms matched text by using a replacement string.

With the complete pattern, enter this *replacement* on regex101:

```
User \4 performed \5 with result \7
```

Capture groups make the replacement dynamic: each output line reuses values extracted from its corresponding input line.

In Python code, the unambiguous replacement syntax `\g<4>` , `\g<5>` and `\g<7>` can also be used.

# Lookahead

A **lookahead** checks what occurs ahead without consuming those characters.

**Positive lookahead** `(?=pattern)` succeeds when the pattern is present.

Find complete log lines belonging to `anna` and ending with `status=0K` :

```
^(?=.*\buser=anna\b)(?=.*status=0K$).+$
```

**Negative lookahead** `(?!pattern)` succeeds when the pattern is absent.

Find log lines that do not end with `status=0K` :

```
^(?!.*status=0K$).+$
```

# Operator Scope and Precedence

Rather than memorising a long precedence table, first identify the **scope** of each operator:

1. Parentheses explicitly create a group.
2. Quantifiers apply to the immediately preceding element or group.
3. Adjacent elements are concatenated.
4. Alternation `|` separates complete alternatives unless parentheses restrict its scope.
5. Anchors and lookarounds assert conditions at the current position.

Compare:

- `ab|cd` : This means `ab` or `cd` .
- `a(b|c)d` : This means `abd` or `acd` .

When in doubt, use parentheses to make the intended structure visible.

# Python re: Main Functions

Python's built-in `re` module provides support for regex:

| Function                    | What it does                                 |
|-----------------------------|----------------------------------------------|
| <code>re.search()</code>    | finds the first match anywhere in the string |
| <code>re.fullmatch()</code> | checks whether the entire string matches     |
| <code>re.findall()</code>   | returns all non-overlapping matches          |
| <code>re.finditer()</code>  | yields match objects one at a time           |
| <code>re.sub()</code>       | replaces matches                             |
| <code>re.split()</code>     | splits text at matches                       |
| <code>re.compile()</code>   | creates a reusable regular-expression object |

For line-oriented input, `re.MULTILINE` makes `^` and `$` apply to each line:

```
matches = re.findall(r"^.*status=OK$", log_text, re.MULTILINE)
```

# Complete Python Example

```
import re

pattern = re.compile(
    r"^(\d{4}-\d{2}-\d{2})\s+"
    r"(\d{2}:\d{2}:\d{2})\s+"
    r"(INFO|WARNING|ERROR)\s+"
    r"user=(\w+)\s+action=(\w+)\s+"
    r"ip=(\d{1,3}(\?:\.\d{1,3}){3})\s+"
    r"status=(OK|NOT_OK|FAILED)$",
    re.MULTILINE,
)

for match in pattern.finditer(log_text):
    date, time, level, user, action, ip, status = match.groups()
    print(user, action, status)
```

The same pattern first explored interactively on regex101 can therefore be reused directly in Python.

# Final Challenge

Using the same log, write an expression that finds lines satisfying all these conditions:

1. the year is `2026` ;
2. the level is `INFO` or `ERROR` ;
3. the username is not `guest` ;
4. the line ends with `status=OK` .

Jurafsky, D. & Martin, J. H., *Speech and Language Processing*, 3rd ed. draft:

- Words: 2.1, 5.1, 2.2, 2.3
- Regular expressions: 2.6
- Text normalization: 2.5, 2.4, 2.8
- Linguistic annotation: 17.1, 17.2, 17.3, 17.6, 18.1, 18.2, 19.1